



# GDB debugging workshop

Session 5



## Sessions 1+2 recap

- Basic GDB CLI usage and commands
- Program data display
- Breakpoints
- GDB TUI
- Debug symbols
- VSCode and GDB
- Advanced breakpoints
  - Conditional
  - Watchpoints
  - Catchpoints



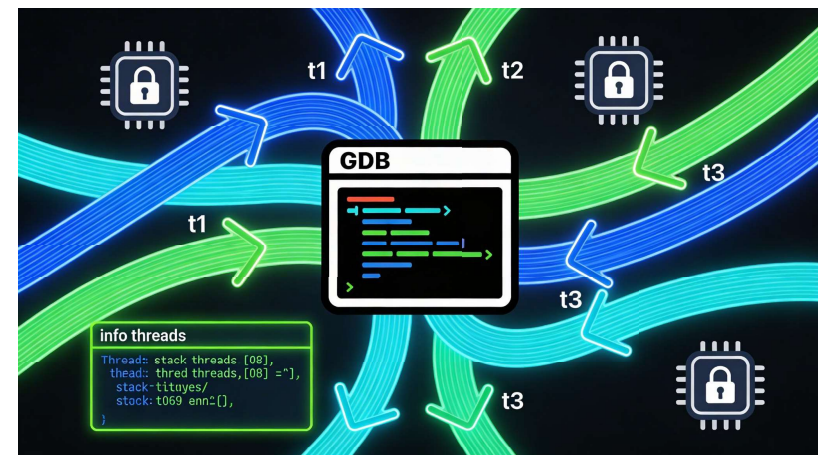
## Sessions 3+4 recap

- Coredump analysis
- Attaching to a live process
- Signals
- Remote debugging
- Altering execution
- Checkpoints
- Tracepoints
- Memory debugging



## Session 5 agenda

- Multithreading
- GDB customization





## Why debugging multi-threaded code is hard?

- **Race conditions** - Non-deterministic timing makes bugs appear randomly
- **Deadlocks** - Threads waiting indefinitely for locks, debuggers miss these
- **Data corruption** - Unsynchronized access to shared memory from multiple threads
- **Timing sensitivity** - Adding debug output changes thread scheduling (Heisenbug)
- **Complex state** - Track multiple execution contexts simultaneously, not just call stacks



## Multi-threading essential commands in GDB

- **info threads** - List all threads with IDs, states (running/stopped), and entry points
- **thread <id>** - Switch context between threads to inspect registers, stack, variables, etc.
- **thread apply all <command>** - Execute command across all threads
- **break file.cpp:42 thread 3** - stops only thread 3 at line 42



## A practical workflow

- **Stop at breakpoint** - All threads halt, use **info threads** to see which threads are where
- **Examine thread state** - **thread <id> → bt → info locals/info args**
- **Check for locks** - Inspect mutex/lock variables directly
- **Step selectively** - Use scheduler locking to advance one thread while others freeze
- **Identify the culprit** - Compare stack traces across threads to find synchronization bugs or deadlocks



## Multithreading - examples

[day03/01\\_multithreading/01\\_basic\\_example](#) (CLI)

[day03/01\\_multithreading/02\\_deadlock](#) (VSCode)



## **Threads behavior on breakpoint hit and resume**



## A thread hits a breakpoint, what happens to other threads?

- All-stop mode - `set non-stop off`
- Non-stop mode - `set non-stop on`

Cannot be changed during program run!



## All-stop mode

On breakpoint - stops all threads

- **Consistent state:** Examine full program snapshot across all threads safely
- **Simple model:** Stable state during inspection  
(**thread N, info locals, info variables** works predictably)
- **Default behavior:** Usually, what users expect

On resume, step-over, step-into - all threads resume by default!

\* Unless otherwise set with gdb command **set scheduler-locking on**



## Non-stop mode

On breakpoint - Only the thread that hit the breakpoint stops, others continue.

When to use:

- **Live systems:** Debug without freezing real-time threads or event handlers
- **Background threads:** Examine crashed thread while others stay responsive (e.g. for network/disk/user I/O)



## On resume (all-stop mode)

**set scheduler-locking <on/replay/step/off>**

- **replay** == scheduler locked in replay mode and unlocked during normal execution.
- **off** == no locking (threads may preempt at any time)
- **on** == full locking (no thread except the current thread may run)  
This applies to both normal execution and replay mode.
- **step** == scheduler locked during stepping commands (step, next, stepi, nexti).  
In this mode, other threads may run during other commands.  
This applies to both normal execution and replay mode.



| Summary table |          | scheduler-lock state => resume behavior |                                 |                                                                                 |
|---------------|----------|-----------------------------------------|---------------------------------|---------------------------------------------------------------------------------|
|               |          | off                                     | on                              | step                                                                            |
| Stop mode     | All-stop | All threads resume                      | Only the current thread resumes | Only the current thread resumes on step/next(i), all threads resume on continue |
|               | Non-stop | N/A                                     | N/A                             | N/A                                                                             |
|               |          | (All threads running anyway)            |                                 |                                                                                 |



## All-stop/non-stop - example

[day03/01\\_multithreading/01\\_basic\\_example](#)

# Debugging race conditions





## **Race conditions vs. Data races**



## Race conditions and Data races (1)

A **race condition** happens when the order of operations across threads is non-deterministic. In some cases, the program may be valid regardless of the actual order of operations (valid race condition). If the program correctness is dependent on the timing or ordering of events, then we may have a *bad* race condition.

A **data race** is a memory access problem: two or more threads access the same memory location, at least one for writing, without synchronization.



## Race conditions and Data races (2)

### Possible issues:

- **Non-deterministic behavior** - Produce different results
- **Hard to reproduce** - Adding debug output changes scheduling, hiding the bug (Heisenbug)

**Symptoms:** Random crashes, corrupted data, deadlocks, logic bugs



## Race conditions and Data races (3)

Debugging race conditions (which are not data race) can be done with all-stop mode, advancing the threads to generate different possible ordering of execution.

For data races, we have special tools:

- Helgrind
- TSAN

# Exercise





## Exercise 1

[exercises/day03/ex01/README.md](#)

Also at: <https://coliru.stacked-crooked.com/a/e7d077ba2dbedffa>

A simple and very clear data race.

We want to use GDB to simulate deliberate context switches to get the lowest value possible for sum.



## Exercise 2a

[exercises/day03/ex02a/README.md](#)

[https://compiler-explorer.com/z/PdEjvG44P](#)



## Exercise 2a

[exercises/day03/ex02a/README.md](#)

[https://compiler-explorer.com/z/PdEjvG44P](#)



## Exercise 2b

[exercises/day03/ex02b/README.md](#)

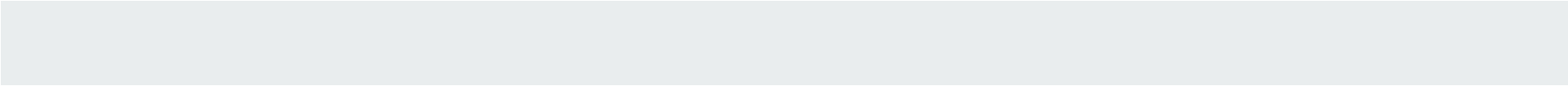
[https://compiler-explorer.com/z/5K5aEWa9E](#)



## Exercise 2c

<exercises/day03/ex02c/README.md>

<https://compiler-explorer.com/z/hYqMx9vjb> ⇐ compiled with TSAN


**Are we all going to Hell(grind)?**

Let's meet *Helgrind*



## Helgrind (1)

- **What it is** - Part of Valgrind suite for detecting thread synchronization errors
- **Detection mechanism** - Tracks memory accesses and builds happens-before dependency graph between threads
- **Race detection** - If two thread accesses have no synchronization path between them, and one is for write, Helgrind reports a race



## Helgrind (2)

- **Reads are safe** - No warnings for concurrent reads (only writes trigger races)
- **Works even on a single-core** - Detects races even with time-sliced threads
- **Also detects** - Potential deadlocks, lock-order violations, API misuse



## How it works?

- **Happens-before graph** - Models synchronization points (mutexes, semaphores, condvars, pthread\_join)
- **Memory tracking** - Records every read/write and which thread performed it
- **Race detection logic** - Two accesses same address, different threads, at least one write, no happens-before path = RACE
- **Conservative approach** - May report false positives if synchronization isn't explicit (e.g., custom spinlocks)
- **Thread state tracking** - Monitors thread creation, destruction, lock acquire/release events



## Basic usage

- Run Helgrind - `valgrind --tool=helgrind <app>`
- Output includes - Thread creation locations, conflicting accesses, stack traces, memory addresses
- Verbose output
  - `-v` flag for detailed traces
  - `--history-level=approx` for speed vs. accuracy trade-off



## Example

[day03/01\\_multithreading/03\\_helgrind](#)  
[exercises/day03/ex01](#)



## Helgrind suppressions and annotations

- Suppression syntax - `--suppressions=file.supp` ([Nice walkthrough](#))
- Annotate code - `ANNOTATE_HAPPENS_BEFORE()` / `ANNOTATE_HAPPENS_AFTER()` for custom locks
- Ignore patterns - `--error-exitcode=0` to ignore (not recommended for production)



## Inline code annotations (1)

**ANNOTATE\_HAPPENS\_BEFORE(addr) / ANNOTATE\_HAPPENS\_AFTER(addr)**

Helgrind annotations that can be sprinkle into the code to *manually* tell the race detector “from here onward, treats accesses to \*addr as ordered by the happens-before relation.”

Inside Helgrind, a **data race** is detected when two accesses to the same address are *not ordered* by a happens-before edge.

These annotations allow to **fake** such an ordering for custom synchronization patterns that the tool would otherwise see as a race.



## Inline code annotations (2)

- **ANNOTATE\_HAPPENS\_BEFORE(addr)**

Inserted **after** a thread accesses \*addr. It tells Helgrind:

“All accesses to \*addr that this thread does *before* this annotation happened before any later access to \*addr that another thread will annotate with ANNOTATE\_HAPPENS\_AFTER(addr).”

- **ANNOTATE\_HAPPENS\_AFTER(addr)**

Inserted **before** an access in another thread. It tells Helgrind:

“The next access to \*addr in this thread should be considered to have happened *after* the last ANNOTATE\_HAPPENS\_BEFORE(addr) done on the same object.”



## Possible use cases

- Node-based queues
  - A “put” thread does `ANNOTATE_HAPPENS_BEFORE(node)` after enqueueing
  - A “get” thread does `ANNOTATE_HAPPENS_AFTER(node)` before dequeueing.
- Thread-safe reference counting



## Helgrind common false positives

- **Vtable pointers** - Helgrind may warn on vptr reads (shared, but safe).
- **Custom synchronization** - Spinlocks, atomic compare-and-swap loops may not be recognized.
- **Third-party libraries** - glibc, pthread, Qt internals may have known (valid) races.



## ThreadSanitizer - TSAN



## ThreadSanitizer - TSAN (1)

- **What is it** - Google's data race detector built into Clang/GCC, part of the Sanitizers family
- **How it works** - Instruments code to track **every** memory access and lock operation
- **Detection mechanism** - Monitors which threads access which memory, detects unsynchronized reads/writes in real-time



## ThreadSanitizer - TSAN (2)

- **Requires a dedicated compilation** - not what we use in production
- **Supports** - C and C++, pthreads, `std::thread`, `atomics`



## Compilation and usage

- **Compile with TSAN** - `-fsanitize=thread` flag
- **Use -O1 or higher** - Overhead is 5-20×, optimization helps  
-O0 makes TSAN unusable (too slow, too much noise)
- **Run normally** - Detects races during execution
- **Output to stderr** - Race reports printed in real-time, stack traces show exact locations



## Example

[day03/01\\_multithreading/04\\_tsan](#)  
[exercises/day03/ex01](#)



## Tools comparison

| Aspect          | TSAN                 | Helgrind         |
|-----------------|----------------------|------------------|
| Compile         | Separate compilation | No recompilation |
| Speed           | 5-20× slower         | 5-50× slower     |
| False Positives | Few                  | More             |
| Best For        | Daily dev            | Deep analysis    |

**Key takeaway:** TSAN is **first line of defense** (development & CI), Helgrind is **deep analysis** (analyze issue)



## **Debugging multithreaded program coredump**



## Debugging multithreaded program coredump

- **View all threads** - `info threads` shows all thread states at crash moment
- **All backtraces** - `thread apply all bt` displays call stack for all threads
- **Full context** - `thread apply all bt full` includes local variables
- **Single thread** - `thread N → bt` to focus on specific thread's stack
- **Find culprit** - Compare backtraces and identify which thread crashed
- **Gotcha** - Even if threads are given names, they are not always displayed in the dump - **You cannot always trust metadata**



## Multithreaded coredump - demo

[day03/01\\_multithreading/05\\_mt\\_coredump](#)





## Controlling GDB initialization

### **Purpose:**

Customize behavior, automate commands, set breakpoints, define macros, etc.



## Controlling GDB initialization - How?

Configuration file (`.gdbinit`) loaded automatically by GDB at startup

- **Location:** `~/.gdbinit` (home directory) or `./gdbinit` (project directory)
- **Priority order:** Project `.gdbinit` → Home `~/.gdbinit` → System defaults
- **Execution:** Commands executed silently before interactive prompt appears



## Common Usages (1)

- **Display settings:**
  - set print pretty on
  - set print array on
  - set pagination off
- **Breakpoint automation:** Pre-set breakpoints at startup for locations
- **Custom commands:** Define macros (shortcuts) using the define keyword



## Common Usages (2)

- **Symbol loading:**

- `add-symbol-file <file> # load symbols from specific file`

- **History & logging:**

- `set history save on`

- `set history size 10000`

- `set logging file debug.log`

- **Architecture-specific: set endian big**



## Passing arguments to macros

- `argc` is number of arguments, `argv0`, `argv1`, ... are the macro arguments
- Example:

```
print $argc
if $argc > 0
    print $arg0
end
if $argc > 1
    print $arg1
end
```



# GDB Hooks



## GDB Hooks (1)

The hook mechanism lets the user define custom command sequences that automatically run before or after specific GDB commands (or every program stop), streamlining repetitive debugging tasks

- `hook-CMD`: Executes **before** the command (pre-execution).
- `hookpost-CMD`: Executes **after** the command (post-execution).



## GDB Hooks (2)

- Hooks work on base command names (e.g., **backtrace**, not **bt**).
- For multi-word commands, hook the last word (e.g., hook-remote for target remote).
- Hooks can call their own command without recursion (GDB prevents re-execution).
- Errors in hooks stop execution and return to the prompt.



## GDB Hooks (3)

A special pseudo-command `stop` triggers on every program halt (breakpoints, watchpoints, step completion):

```
define hook-stop
  echo "Stopped at: " $pc
  info registers
end
```



## Hooks - demo

[day03/02\\_gdb\\_customizations/05\\_hooks](#)



## External sources

- Setting code search paths: *directory* command to add source lookup paths
- Loading external code: *source* <external-plugin>

Both can be used as initialization commands in **.gdbinit**

Or as GDB CLI commands



## source vs. python exec()

- **Python initialization:** Load custom Python scripts with source script.py  
`python exec(open(' script.py ').read())`
- And other Python commands...

<https://sourceware.org/gdb/current/onlinedocs/gdb.html/Python-Commands.html>



## **.gdbinit usage example**

[day03/02\\_gdb\\_customization/01\\_gdbinit](#)



## Possible gotcha

**The issue:** a GDB security feature designed to prevent malicious scripts from automatically running when entering a directory, resulting by failing to load the initialization script.

**The fix:** In top-level .gdbinit add

```
add-auto-load-safe-path <location of gdbinit>
```



## Customizing GDB with Python

- **GDB Python API:** Full Python interpreter embedded in GDB for extending functionality
- **Access:** Available by default in most GDB builds
- **Use cases:** Custom commands, dynamic breakpoints, data visualization, automation
- **Execution methods:** Interactive prompt, source scripts, inline scripts in **.gdbinit**
- **Version compatibility:** Both Python 2.7 and Python 3.x



## Customizing GDB with Python - usage examples

- **Single Python command:** `python <valid Python command>`

Example:

```
python print("test")
```

- **Loading Python script:**

```
python exec(open('script.py').read())    – Or –  
source script.py
```



## Writing custom GDB Python commands (1)

- **Define command class:** Inherit from `gdb.Command` with `invoke()` method implementation
- **Command registration:** Register in module `__init__` (or call constructor)
- **Access GDB state:** Query breakpoints, threads, frames, variables via `gdb.breakpoints()`, `gdb.selected_frame()`



## Writing custom GDB Python commands (2)

- **Parse arguments:** Extract parameters from args string in `invoke()` method
- **Output:** Use `gdb.write()` for console output instead of `print()`
- **Error handling:** Raise `gdb.GdbError()` for proper error propagation to GDB console

Full API reference: <https://sourceware.org/gdb/current/onlinedocs/gdb.html/Python-API.html>



## Adding GDB custom command - example

[day03/02\\_gdb\\_customizations/02\\_python](#)

[day02/05\\_tracepoints](#)



## **Yet another GDB UI?**



## Yet another GDB UI?

- **What is GDB Dashboard?** TUI (Text User Interface) replacement - modern, Python-based alternative to `gdb -tui`
- **Repo:** <https://github.com/cyrus-and/gdb-dashboard>
- **Installation:** Clone repo, source `gdbinit` in your `.gdbinit` or project config
- **Why use it?** Superior layout, real-time updates, highly customizable, cleaner than built-in TUI



## GDB dashboard demo

[day03/02\\_gdb\\_customizations/03\\_dashboard](#)



## GDB Enhanced Features (GEF)

- **What is GEF?** Advanced GDB plugin using Python API (Pronounced *dʒɛf* - "Jeff")
  - Designed for exploit developers, reverse engineers, dynamic analysis
- **Repo:** <https://github.com/hugsy/gef>
  - Actively maintained
  - 50+ commands
  - Architecture agnostic
- **Target users:** Exploit development, reverse engineering, embedded debugging
- **Key advantages:** No external dependencies, fast execution, full Python 3 support
- **Installation:** One-liner `bash -c "$(curl -fsSL https://gef.blah.cat/sh)"`



## GDB Enhanced Features (GEF)

<https://github.com/hugsy/gef/blob/main/docs/index.md>



## Other UI Python Customizations

| Plugin | Description                                                                               | Best For                         | GitHub Stars (approx)                  |
|--------|-------------------------------------------------------------------------------------------|----------------------------------|----------------------------------------|
| Pwndbg | Feature-rich TUI with heap analysis, exploit helpers, colored output. Supports Python 2/3 | Exploit dev, low-level debugging | 8k+                                    |
| PEDA   | Lightweight Python exploit assistant with ROP tools, pattern generation                   | Quick exploit prototyping        | 4k+                                    |
| GEP    | Enhanced prompt with fzf history/search, autosuggestions, tab completion                  | Interactive command-line UX      | Recent, growing <a href="#">github</a> |

Top GDB plugins for 2025 - <https://markaicode.com/top-gdb-plugins-2025/>



## Session 5 summary

- Multithreading
- GDB customization



## Towards session 6

- Reverse execution
- Complementary tools
- What's new in C++ in debugging context?